National Telecommunication Institute (NTI)

UART Full Duplex Project

Part 1: Transmitter Using Verilog HDL

Part 2: Receiver and Transmitter Using SystemVerilog

Prepared by: Nada Hosny

Under the Supervision of:
Eng. Mohamed Elsayed
Eng. Moaz Khaled

Email: nada.hosny300@gmail.com

LinkedIn: linkedin.com/in/nada-eldeeb-0b5400264

August 2026

1 Introduction

UART (Universal Asynchronous Receiver-Transmitter) is a serial communication protocol used to transmit data between digital systems without a shared clock.

This project implements an **8-bit UART Transmitter** using Verilog HDL. The transmitted frame consists of a Start bit, Data bits, an optional Parity bit, and a Stop bit.

2 Requirements

The main requirements of the UART Transmitter are:

- Receive new data on **P_INPUT** only when **V_INPUT** is high.
- **V_INPUT** is asserted for one clock cycle only.
- Use an asynchronous active-low **RESET** to clear all registers.
- **BUSY** remains high during frame transmission and low otherwise.
- UART_TX cannot accept new data while transmitting.
- **TX_OUTPUT** remains high during the IDLE state.
- **P_EN** controls the parity bit:
  - 0: Parity disabled.
  - 1: Parity enabled.
- **P_BIT** selects the parity type:
  - 0: Even parity.
  - 1: Odd parity.

3 Specifications

The UART TX module consists of the following input and output signals:

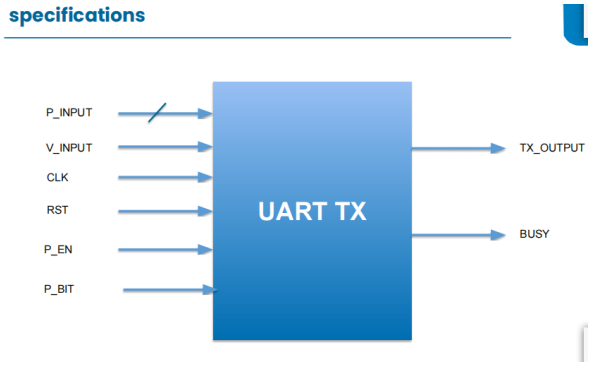


Figure 1: UART TX Block Diagram

UART TX converts parallel input data into a serial output stream.

4 UART TX Architecture

Figure 2 shows the overall design of the UART Transmitter, including the main input and output signals and the internal modules.

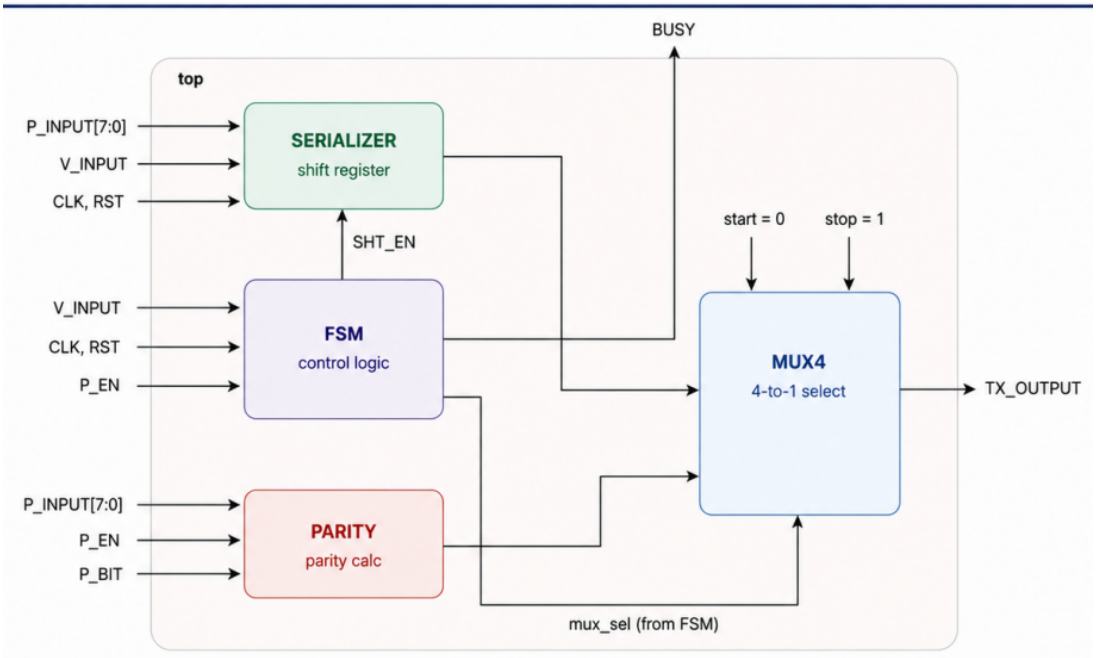


Figure 2: Top-level architecture of the UART Transmitter.

The design shows the input and output signals and their connections to the main UART TX modules.

5 Serializer

The Serializer takes the **8-bit parallel data** and converts it into **serial data**. The **least significant bit (LSB)** is shifted first.

The data width is defined using a parameter **N**, so the Serializer can be easily modified to support different data sizes in the future.

5.1 Serializer Design

The following shows the Verilog implementation of the Serializer:

```
E:/arc/serilizar.v - Default
Ln#
 1  module serializer #(
 2      parameter N = 8
 3  )(
 4      input [N-1:0] P_INPUT,
 5      input V_INPUT,
 6      input SHT_EN,
 7      input CLK,
 8      input RST,
 9      output SERIAL_OUT
10  );
11
12  reg [N-1:0] shift_reg;
13
14  always @(posedge CLK or negedge RST)
15  begin
16
17      if(RST==0) begin
18          shift_reg <= {N{1'b0}};
19      end
20
21      else if(V_INPUT==1) begin
22          shift_reg <= P_INPUT;
23      end
24
25      else if(SHT_EN==1) begin
26          shift_reg <= {1'b0, shift_reg[N-1:1]};
27      end
28
29  end
30
31  assign SERIAL_OUT = shift_reg[0];
32
33  endmodule
34
```

Figure 3: Serializer Verilog Code

5.2 Serializer Verification

A testbench was built to verify the Serializer using three main cases:

- **Reset:** RST = 0 to check that the shift register is cleared.
- **Valid Data:** V_INPUT = 1 and SHT_EN = 1 to verify the correct data path.
- **Invalid Data:** Data was applied with V_INPUT = 0 and SHT_EN = 0 to confirm that no new data is loaded.

The simulation output shows that the Serializer works correctly and shifts the data starting from the LSB.

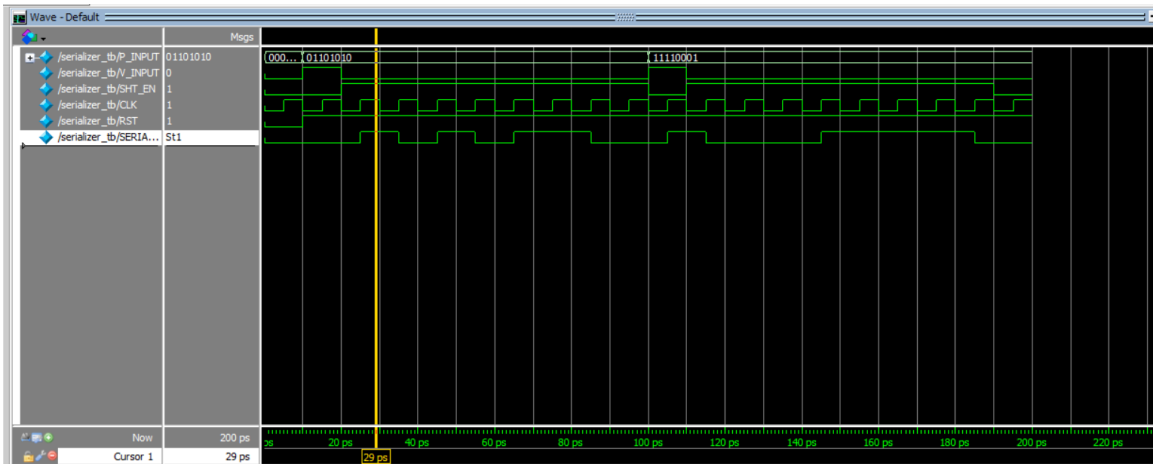


Figure 4: Serializer Testbench Simulation

6 Parity Module

Parity is used to add one extra bit to the data for simple error detection. The parity bit depends on the number of 1's in the input data.

Original Data	Even Parity	Odd Parity
0 0 0 0 0 0 0 0	0	1
0 1 0 1 1 0 1 1	1	0
0 1 0 1 0 1 0 1	0	1
1 1 1 1 1 1 1 1	0	1
1 0 0 0 0 0 0 0	1	0
0 1 0 0 1 0 0 1	1	0

Figure 5: Parity Concept

The module counts the number of 1's in the input data and checks whether the count is even or odd. Then, the required parity value is generated.

6.1 Parity Design

Inputs: P_INPUT, P_EN, P_BIT

Output: PARITY_BIT

The module uses a counter to count the number of 1's in **P_INPUT**. The count is then compared to determine whether the number of 1's is even or odd.

```
.n#
1  module parity #(parameter N = 8)(
2      input [N-1:0] P_INPUT,
3      input P_EN,
4      input P_BIT,
5      output reg PARITY_BIT
6  );
7
8  integer i;
9  integer count_ones;
10
11 always @(*) begin
12
13 count_ones = 0;
14 PARITY_BIT = 0;
15 for(i = 0; i < N; i = i + 1) begin
16   if(P_INPUT[i] == 1)
17       count_ones = count_ones + 1;
18   end
19
20 if(P_EN == 1) begin
21    if(P_BIT == 0) begin
22       if(count_ones % 2 == 0)
23            PARITY_BIT = 0;
24       else
25            PARITY_BIT = 1;
26    end
27    else begin
28      if(count_ones % 2 == 0)
29           PARITY_BIT = 1;
30      else
31           PARITY_BIT = 0;
32        end
33
34      end
```

Figure 6: Parity Module Verilog Code

The four possible cases are:

- **P_EN = 0:** Parity is disabled.
- **P_EN = 1, P_BIT = 0:** Even parity is selected.
- **P_EN = 1, P_BIT = 1:** Odd parity is selected.
- For even number of 1's, the even parity bit is **0** and the odd parity bit is **1**. For odd number of 1's, the values are reversed.

6.2 Parity Verification

A testbench was built to test different input data and parity settings. The simulation shows that the **PARITY_BIT** changes correctly according to the input data and the selected parity type.

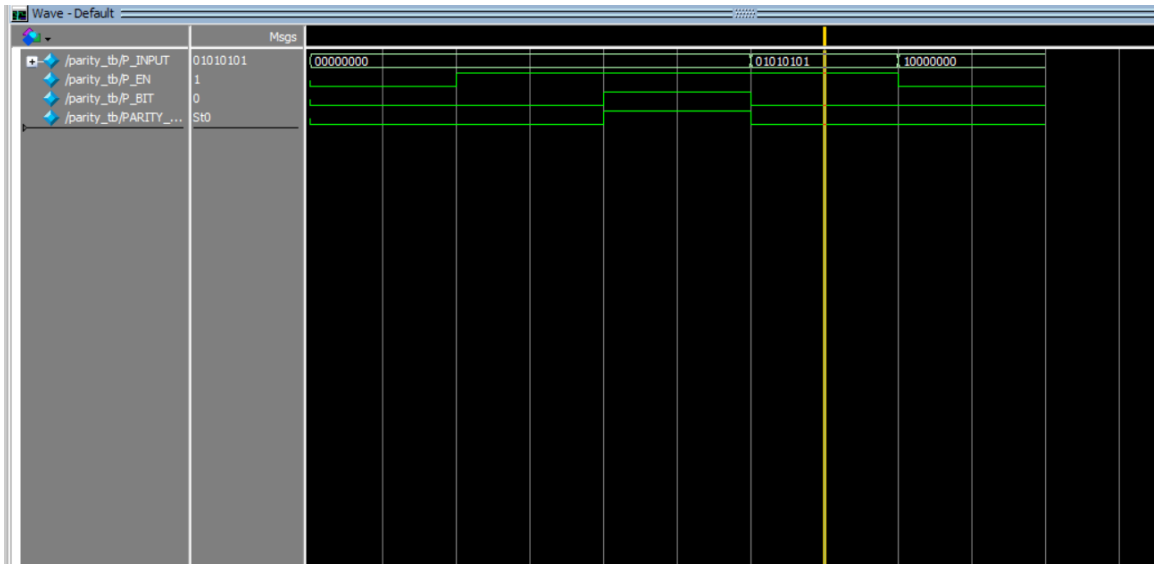


Figure 7: Parity Module Testbench Simulation

7 Finite State Machine (FSM)

The FSM controls the Serializer and Parity modules by determining which block works and when. A **Moore FSM** is used, where the outputs depend only on the current state.

The UART frame is transmitted in the following order:

Start (0) → Data (8 bits) → Parity (optional) → Stop (1)

7.1 FSM Inputs and Outputs

Signal	Description
V_INPUT	Data valid signal
P_INPUT	8-bit parallel input data
CLK	System clock
RST	Asynchronous active-low reset
P_EN	Parity enable
P_BIT	Parity type selection

Table 1: FSM Input Signals

Signal	Description
BUSY	High during transmission
SHT_EN	Serializer enable signal
mux_sel	Selects the transmitted data source

Table 2: FSM Output Signals

7.2 FSM State Diagram

The FSM consists of five states: IDLE, START, DATA, PARITY, and STOP.

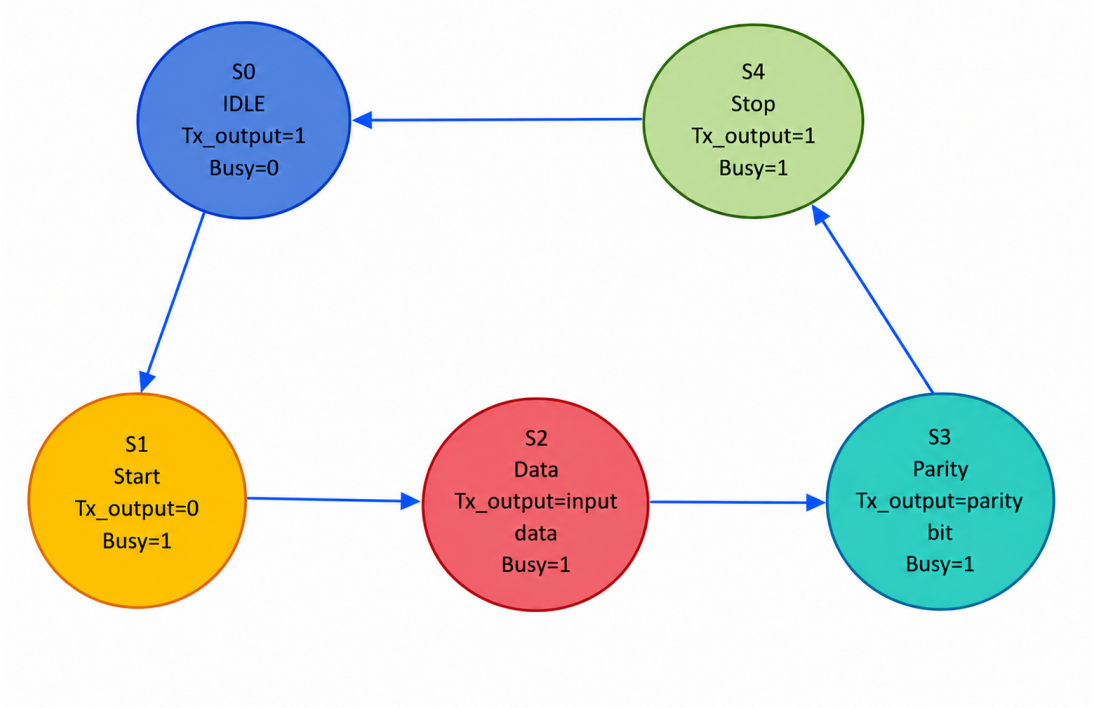


Figure 8: UART Transmitter FSM State Diagram

7.3 State Parameters

The states are defined as follows:

State	Parameter	Description
S0	3'b000	IDLE
S1	3'b001	START
S2	3'b010	DATA
S3	3'b011	PARITY
S4	3'b100	STOP

Table 3: FSM State Parameters

7.4 State Description

State	TX_OUTPUT	BUSY	SHT_EN	mux_sel	Description
S0 IDLE	1	0	0	–	No transmission. Wait for V_INPUT = 1.
S1 START	0	1	0	00	Transmits the Start bit (0) for one clock cycle.
S2 DATA	Data	1	1	01	Serializer is enabled and transmits the 8 data bits, LSB first.
S3 PARITY	Parity	1	0	10	Transmits the parity bit when P_EN = 1.
S4 STOP	1	1	0	11	Transmits the Stop bit (1) for one clock cycle.

Table 4: FSM States and Outputs

7.5 State Transitions

- **S0 → S1:** The FSM moves from IDLE to START when **V_INPUT = 1**, indicating that valid data is available.
- **S1 → S2:** The Start bit is transmitted for one clock cycle, then the FSM moves to the DATA state.
- **S2 → S3/S4:** The FSM waits for all 8 data bits to be transmitted. A counter is used to count the 8 data cycles. If **P_EN = 1**, the FSM moves to S3. Otherwise, it skips the parity state and moves directly to S4.
- **S3 → S4:** The parity bit is transmitted for one clock cycle, then the FSM moves to the STOP state.
- **S4 → S0:** The Stop bit is transmitted for one clock cycle, then the FSM returns to IDLE.

7.6 FSM Implementation

```
module FSM(
input V_INPUT,
input P_EN,
input CLK,
input RST,
output reg BUSY,
output reg SHT_EN,
output reg [1:0] mux_sel
);

// define every state from IDLE to stop in order of the frame
parameter S0 = 3'b000;
parameter S1 = 3'b001;
parameter S2 = 3'b010;
parameter S3 = 3'b011;
parameter S4 = 3'b100;

reg [2:0] state;
reg [2:0] next_state;
reg [2:0] count;

always @(posedge CLK or negedge RST)
begin

if (RST == 0)
begin
    state <= S0;
    count <= 0;
end

else
begin
    state <= next_state;
```

(a) FSM definition, state parameters, and registers.

```
    state <= next_state;

    if (state == S2)
        count <= count + 1;
    else
        count <= 0;
end

end

always @(*)
begin

next_state = state;
mux_sel = 2'b00;
SHT_EN = 0;
BUSY = (state == S0) ? 1'b0 : 1'b1;

case(state)

S0: begin
    if (V_INPUT == 1)
        next_state = S1;

    SHT_EN = 0;
end

// then when start bit finish the serializer start work
S1: begin
    next_state = S2;

    SHT_EN = 0;
    mux_sel = 2'b00;
end
```

(b) IDLE, START, and DATA state transitions.

```
// data bits
S2:
begin

SHT_EN = 1;
mux_sel = 2'b01;

// after 8 data bits
if (count == 7)
begin
    if (P_EN == 1)
    begin
        next_state = S3;
    end
    else
    begin
        next_state = S4;
    end
end

end

// parity
S3: begin
    next_state = S4;

    SHT_EN = 0;
    mux_sel = 2'b10;
end

// stop
S4: begin
    next_state = S0;
```

(c) DATA, PARITY, and STOP state control.

Figure 9: FSM Verilog implementation.

7.7 FSM Verification

A testbench was built to verify the FSM states and transitions. Initially, some tests failed due to timing issues between the FSM outputs and the testbench checks.

The problem was fixed by adding the required clock pulse between state transitions and checking each state during its correct clock cycle. **P_EN** was also set before starting the transmission.

```
VSIM 3> run -all
# Reset PASS
# IDLE PASS
# start Failed
# data pass
# parity Failed
# stop failed
# Return IDLE Test PASS
# NO PARITY Test FAIL
# Reset During TX Test PASS
# ** Note: $stop    : E:/arc/FSM_tb.v(156)
#    Time: 295 ps  Iteration: 0  Instance: /FSM_tb
# Break in Module FSM_tb at E:/arc/FSM_tb.v line 156
# WARNING: No extended dataflow license exists

VSIM 4>
```

```
# 6 compiles, 0 failed with no errors.
VSIM 7> vsim -gui work.FSM_tb
# End time: 23:08:40 on Aug 27,2026, Elapsed time: 0:13:4
2
# Errors: 0, Warnings: 1
# vsim -gui work.FSM_tb
# Start time: 23:08:40 on Aug 27,2026
# Loading work.FSM_tb
# Loading work.FSM
add wave -position end sim:/FSM_tb/*
VSIM 9> run -all
# Reset PASS
# IDLE PASS
# start pass
# data pass
# parity Pass
# stop pass
# Return IDLE Test PASS
# NO PARITY Test PASS
# Reset During TX Test PASS
# ** Note: $stop    : E:/arc/FSM_tb.v(152)
#    Time: 275 ps  Iteration: 0  Instance: /FSM_tb
# Break in Module FSM_tb at E:/arc/FSM_tb.v line 152
```

Figure 10: FSM testbench before and after fixing the timing issue.

The final waveform confirms the correct FSM operation and state transitions.

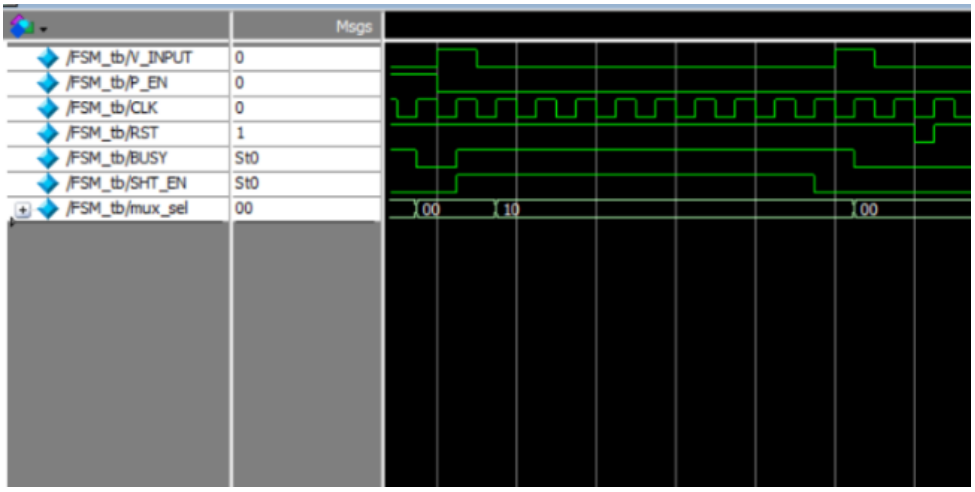


Figure 11: FSM simulation waveform.

7.8 MUX Module

The MUX is a **4-to-1 multiplexer** that selects the required signal for **TX_OUTPUT** according to the **mux_sel** value.

```
module MUX4(
input start,
input stop,
input SERIAL_OUT,
input PARITY_BIT,
input [1:0] mux_sel,
output reg Tx_output
);
always @(*) begin
case(mux_sel)
2'b00:
    Tx_output = start;
2'b01:
    Tx_output = SERIAL_OUT;
2'b10:
    Tx_output = PARITY_BIT;
2'b11:
    Tx_output = stop;
endcase
end
endmodule
```

Figure 12: MUX Verilog Implementation

The selection is:

- **00**: Start bit
- **01**: Serializer output (Data)
- **10**: Parity bit
- **11**: Stop bit

The MUX output is connected to **TX_OUTPUT** to transmit the selected part of the UART frame.

7.9 Top Module

The top module integrates the **FSM, Serializer, Parity, and MUX** modules to complete the UART transmitter design. It controls the transmission flow and generates the final **TX_OUTPUT**.

7.10 Top Module Verification

A testbench was built to verify the complete UART transmitter by testing reset, data transmission, parity, and reset during transmission.

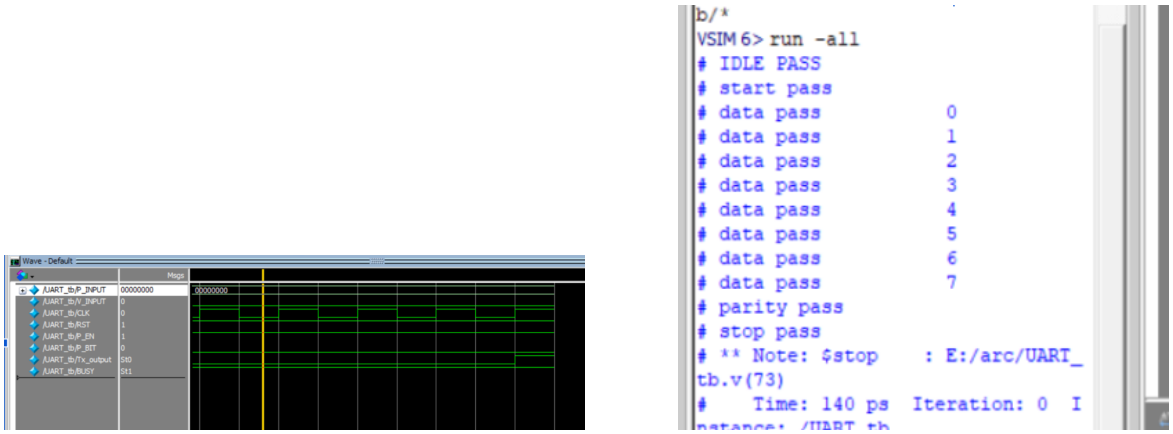
Test Case	Purpose
Reset	Check the system reset operation
00000000	Test all-zero data
11111111	Test all-one data
01010110	Test normal data transmission
P_BIT = 0	Test even parity
P_BIT = 1	Test odd parity
P_EN = 0	Test transmission without parity
P_EN = 1	Test transmission with parity
Parity Test	Verify parity for different input data
Reset Mid-Transmission	Check reset during transmission

Table 5: Top-level UART transmitter test cases.

The testbench checks that the complete UART frame is transmitted correctly and that the system returns to the IDLE state after transmission.

7.10.1 Test Case 1: All-Zero Data

The UART was tested with **P_INPUT = 00000000**, parity enabled, and **P_BIT = 0** for even parity. The testbench confirms the correct transmission of the complete frame.



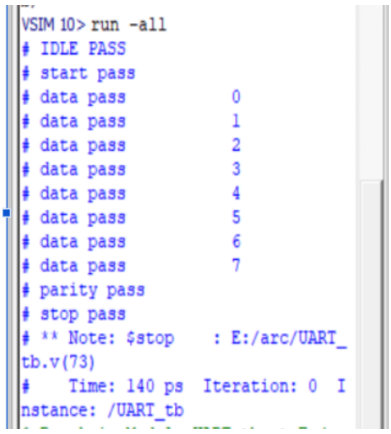
(a) UART transmission waveform.

(b) ModelSim testbench output showing all tests passed.

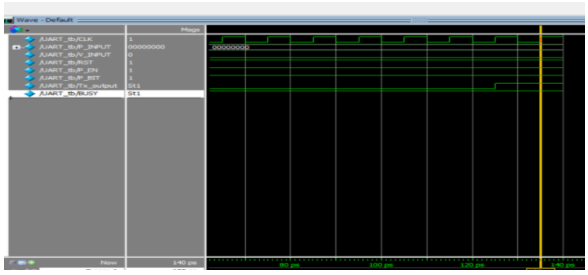
Figure 13: Test Case 1 verification for all-zero data with even parity.

7.10.2 Test Case 2: Odd Parity

The UART was tested with **P_INPUT = 00000000**, parity enabled, and **P_BIT = 1** for odd parity. The testbench confirms the correct transmission of the complete frame.



(a) ModelSim testbench output.



(b) UART transmission waveform.

Figure 14: Test Case 2 verification with odd parity.

7.10.3 Test Case 3: Parity Disabled

The UART was tested with **P_INPUT = 00000000** and **P_EN = 0** to disable the parity bit. The frame is transmitted without the parity bit.

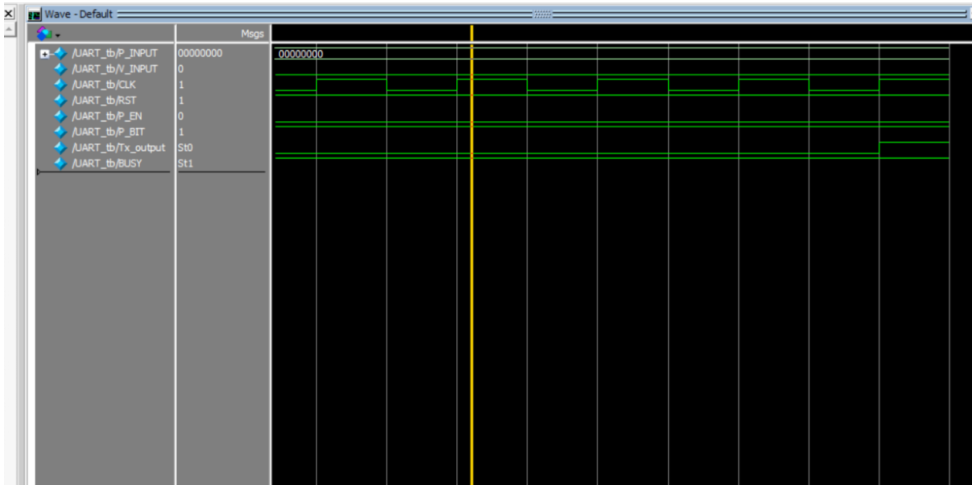
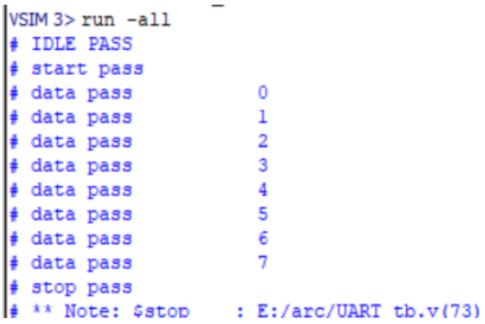


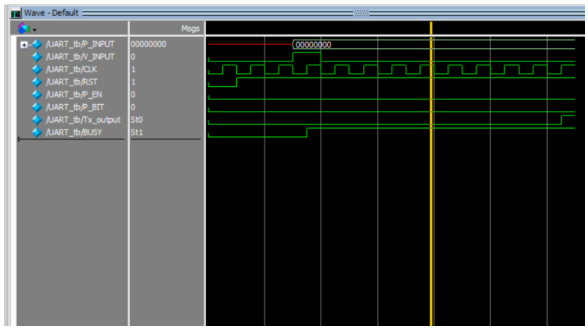
Figure 15: UART transmission waveform with parity disabled.

7.10.4 Test Case 4: Parity Bit = 0

The UART was tested with **P_INPUT = 00000000**, parity enabled, and **P_BIT = 0**. The test confirms that the parity bit can be **0**, the same value as the data bits, while the transmission still works correctly.



(a) Testbench output showing all tests passed.

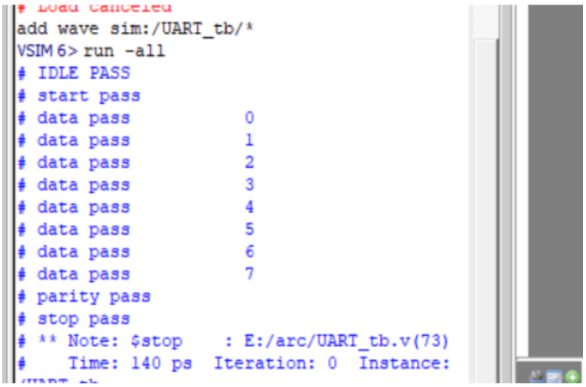


(b) Waveform showing the transmitted frame.

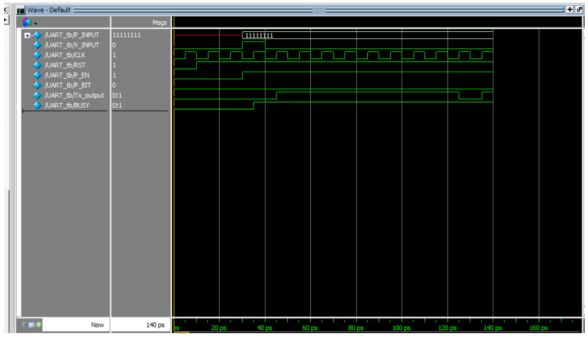
Figure 16: Test Case 4 verification with parity bit equal to 0.

7.10.5 Test Case 5: All-One Data

The UART was tested with **P_INPUT = 11111111**, parity enabled, and **P_BIT = 0** for even parity. The testbench confirms the correct transmission of the complete frame.



(a) Testbench output showing successful transmission.

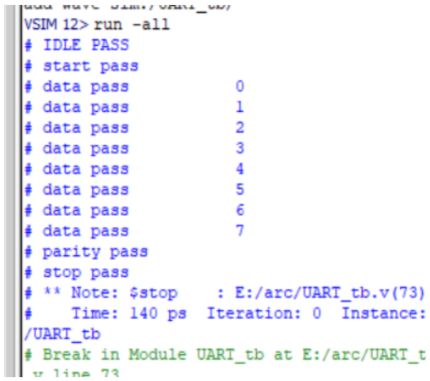


(b) Waveform of the transmitted frame.

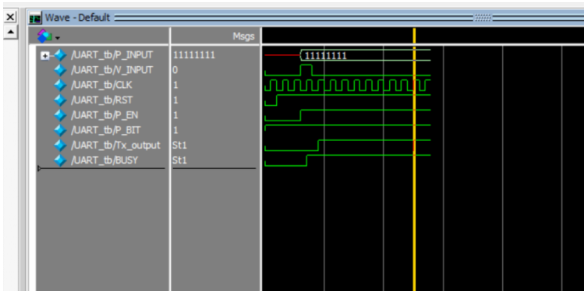
Figure 17: Test Case 5 verification with all-one data and even parity.

7.10.6 Test Case 6: All-One Data with Odd Parity

The UART was tested with **P_INPUT = 11111111**, parity enabled, and **P_BIT = 1** for odd parity. The testbench confirms the correct transmission of the complete frame.



(a) Testbench output showing successful transmission.

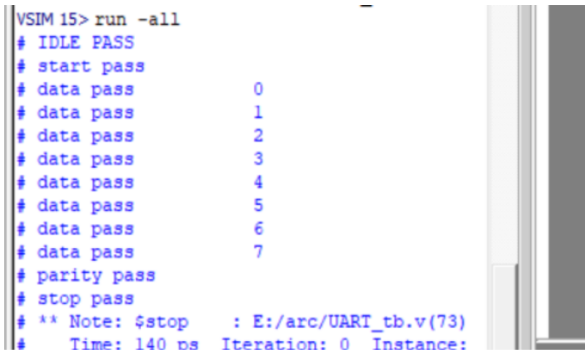


(b) Waveform of the transmitted frame.

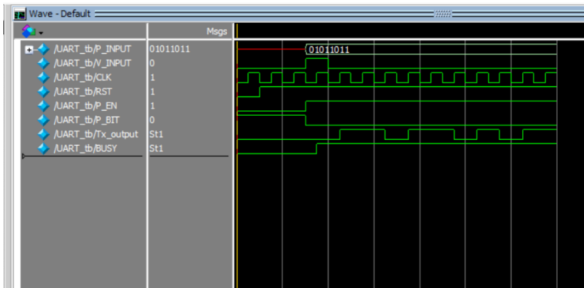
Figure 18: Test Case 6 verification with all-one data and odd parity.

7.10.7 Test Case 7: Normal Data Transmission

The UART was tested with `P_INPUT = 01011011` and parity enabled to verify the complete data transmission path.



(a) Simulation output showing that IDLE, START, DATA, PARITY, and STOP states pass successfully.

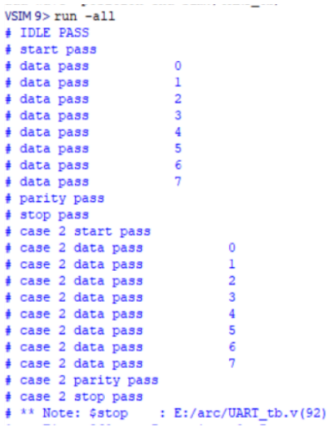


(b) Waveform showing the transmitted data and the corresponding UART control signals.

Figure 19: UART test results for `P_INPUT = 01011011` with parity enabled.

7.10.8 Test Case 8: Sending Two Frames

The UART was tested by sending two consecutive frames with `P_INPUT = 11111111` and `P_INPUT = 00000000` to verify that the system can correctly transmit multiple frames.



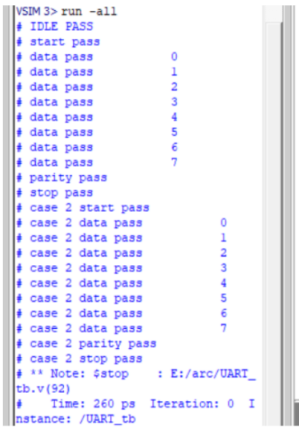
(a) Simulation output showing successful transmission of the two frames.

(b) Waveform showing the two transmitted frames: `11111111` and `00000000`.

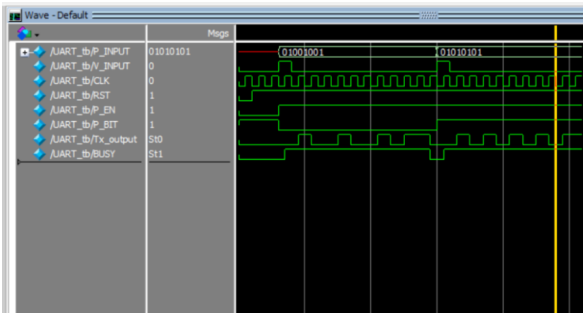
Figure 20: Test Case 8: UART transmission of two consecutive frames.

7.10.9 Test Case 9: Sending Two Normal Data Frames

The UART was tested by sending two consecutive frames with `P_INPUT = 01001001` and `P_INPUT = 01010101`, with parity enabled, to verify correct transmission of different data frames.



(a) Simulation output showing that both frames pass all transmission states successfully.



(b) Waveform showing the transmission of the two data frames.

Figure 21: Test Case 9: UART transmission of two different normal data frames.

7.10.10 Overall UART Testbench Waveform

Figure 22 shows the complete UART simulation waveform for all implemented test cases. The waveform demonstrates the transmission of different input patterns, including all-zero and all-one data, as well as mixed data patterns. Both even and odd parity configurations are tested, along with cases where parity is disabled. The waveform also shows the changes in `V_INPUT`, `P_EN`, and `P_BIT` for each test case. The `Tx_output` signal changes according to the UART frame sequence, while the `BUSY` signal indicates the active transmission period. The reset signal is also included to verify the reset operation of the UART.

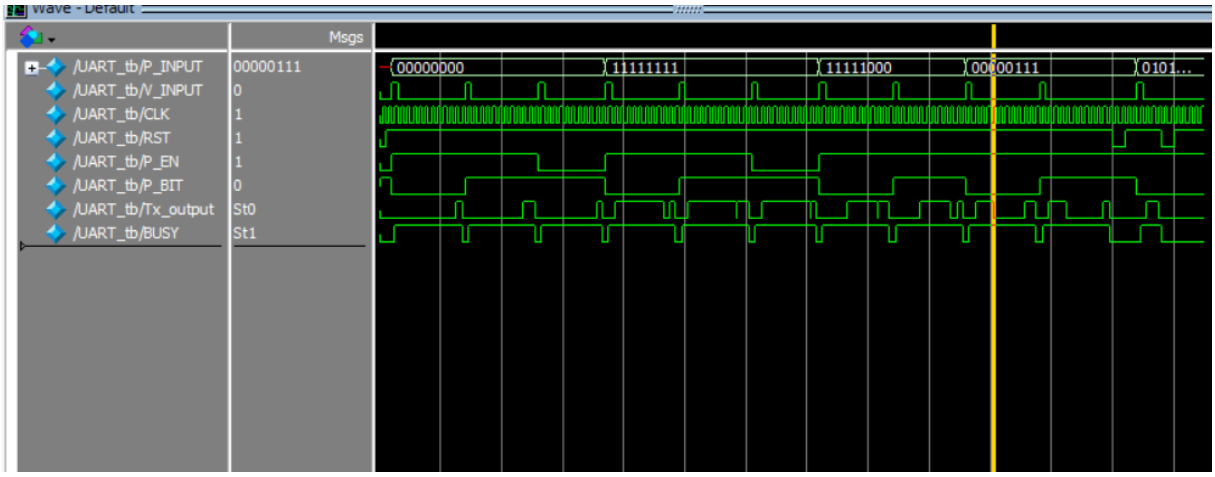


Figure 22: Complete UART testbench waveform for all test cases.

PART 2

UART Receiver and Transmitter

Using SystemVerilog

UART Full Duplex Communication

Receiver and Transmitter Design

Nada Hosny

August 2026

8 UART Receiver

The UART Receiver is responsible for receiving the serial data and converting it back to parallel data. The receiver follows the UART frame rules, starting with the **Start Bit**, then receiving the **8 Data Bits** from LSB to MSB, checking the optional **Parity Bit**, and finally checking the **Stop Bit**.

The main challenge in the Receiver is detecting the correct Start Bit and receiving the data bits at the correct time without confusing normal data transitions with the Start Bit. The Receiver also needs to handle the optional Parity Bit and make sure that the Stop Bit is received correctly.

To make the design easier to understand and test, the Receiver is divided into separate modules.

8.1 Transceiver Verification

First, we made sure that the **UART Transceiver** satisfies all the required specifications and works correctly for the different UART cases.

After that, the name of the **Transceiver** was changed to match the name used in the **Testbench**, to make sure that the design is connected and tested correctly.

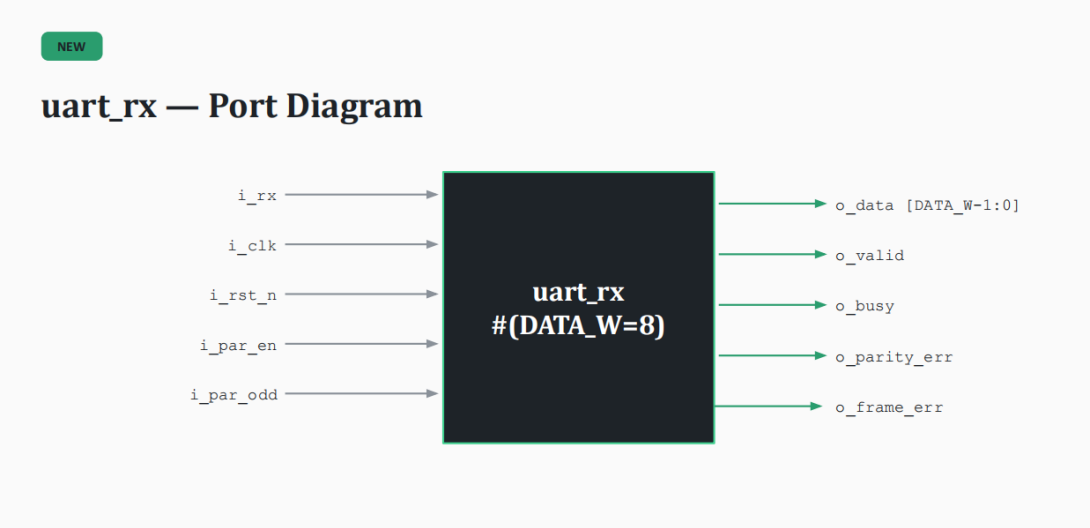


Figure 23: UART Receiver Port Diagram

The UART Receiver receives serial data through **i_rx** and uses the clock, reset, and parity signals to control the reception. The received data is provided through **o_data**, while **o_valid** shows when the data is ready. The receiver also provides **o_busy**, **o_parity_err**, and **o_frame_err** signals for the receiver status and error detection.

8.2 UART Receiver Modules

The UART Receiver is divided into four main modules. Each module has a specific task in the reception process:

- **Start Bit Detector:** Detects the first bit of the UART frame and confirms that a valid Start Bit is received.

- **FSM:** Controls the receiver flow and moves between the different reception states such as Start, Data, Parity, and Stop.
- **Serial-to-Parallel Converter:** Receives the serial data bits one by one and stores them as parallel data.
- **Parity Checker:** Checks the received data and verifies the Parity Bit when parity is enabled.

The four modules work together to receive the complete UART frame and detect any Parity or Frame errors.

8.3 UART Receiver Design

The UART receiver is designed using four main modules that work together to receive and process the incoming serial data. The `DETECTOR` detects the start bit and provides the received bit to the rest of the design. The `FSM` controls the receiving process and generates the required enable signals. The `SER2PAR` module collects the serial data bits and converts them into parallel data. Finally, the `PARITY` module checks the received parity bit and generates the parity error signal.

Figure 24 shows how the four modules are connected together inside the `uart_rx` top module.

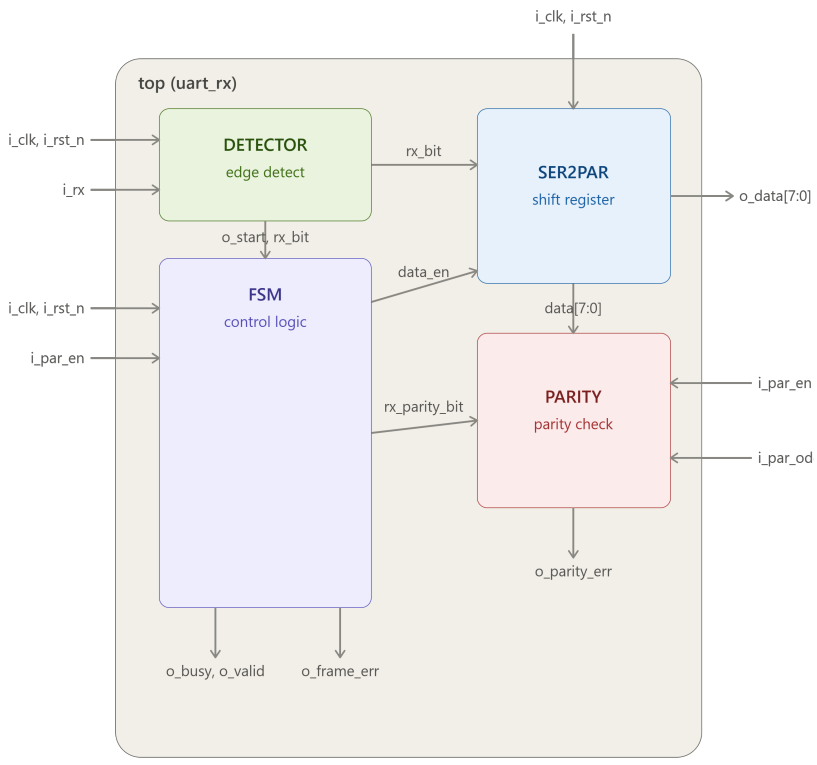


Figure 24: UART Receiver Design Using the Four Main Modules

8.4 Parity Check

The same strategy used in the UART transmitter is used to calculate the expected parity bit at the receiver. The number of ones in the received data is counted, and the expected parity bit is calculated based on the selected parity type.

The calculated parity is then compared with the received parity bit. If both values are equal, the parity is correct and the `o_parity_err` signal is set to 0. If they are different, a parity error is detected and `o_parity_err` is set to 1.

```
module uart_rx_parity #( parameter DATA_W = 8)(
    input logic [DATA_W-1:0] i_data,
    input logic      i_par_en,
    input logic      i_par_odd,
    input logic      i_rx_parity,
    output logic    o_parity_err
);

int i;
int count_ones;
logic expected_parity;

always_comb begin
count_ones = 0;
expected_parity = 0;
o_parity_err = 0;

    for(i = 0; i < DATA_W; i = i + 1) begin
        if(i_data[i] == 1)
            count_ones = count_ones + 1;
    end
    if(i_par_odd == 0) begin
        if(count_ones % 2 == 0)
            expected_parity = 0;
        else
            expected_parity = 1;
    end
    else begin
        if(count_ones % 2 == 0)
            expected_parity = 1;
        else
            expected_parity = 0;
    end

    if(i_par_en) begin
        if(i_rx_parity == expected_parity)
            o_parity_err = 0;
        else
```

Figure 25: UART Receiver Parity Check

8.5 RX Detector

The RX detector is used to detect the start of the UART frame. It stores the previous value of the RX signal in a register called `rx_q`. When the receiver is in the idle state, the RX signal should be 1. If the previous RX value is 1 and the current RX value becomes 0, this means that the start bit has been detected. In this case, `o_start` becomes 1 and the FSM can start receiving the data. The current RX value is also passed through `o_rx_bit` to be used by the other receiver modules.

```
module uart_rx_detector (
    input logic i_clk,
    input logic i_rst_n,
    input logic i_rx,
    input logic i_idle,
    output logic o_start,
    output logic o_rx_bit
);

logic rx_q;

always_ff @(posedge i_clk or negedge i_rst_n) begin
    if (i_rst_n==1)
        rx_q <= 1'b1;
    else
        rx_q <= i_rx;
end

always_comb begin
    o_start = i_idle && rx_q && !i_rx;
    o_rx_bit = i_rx;
end

endmodule
```

Figure 26: RX Detector module

8.6 Serial to Parallel

This module performs the reverse operation of the serializer. It takes the received data in serial form, bit by bit, and converts it into parallel data, where all the received bits are stored together.

```
module uart_rx_ser2par #(parameter DATA_W = 8)(
input logic i_clk,
input logic i_rst_n,
input logic i_data_en,
input logic i_rx_bit,
input logic i_done,
output logic [DATA_W-1:0] o_data
);

int counter;

always_ff @(posedge i_clk or negedge i_rst_n) begin
    if(i_rst_n==1) begin
        o_data <= '0;
        counter <= 0;
    end
    else begin

        if(i_data_en) begin
            o_data[counter] <= i_rx_bit;

            if(counter == DATA_W - 1)
                counter <= 0;
            else
                counter <= counter + 1;
        end

        if(i_done)
            counter <= 0;

    end
end

endmodule
```

Figure 27: Serial to Parallel module

8.7 Receiver FSM

The Receiver FSM controls the UART receiving process using four states: *IDLE*, *DATA*, *PARITY*, and *STOP*. It waits for the start detection, receives the data bits sequentially, handles the parity bit when enabled, and finally checks the stop bit before completing the frame and returning to the *IDLE* state.

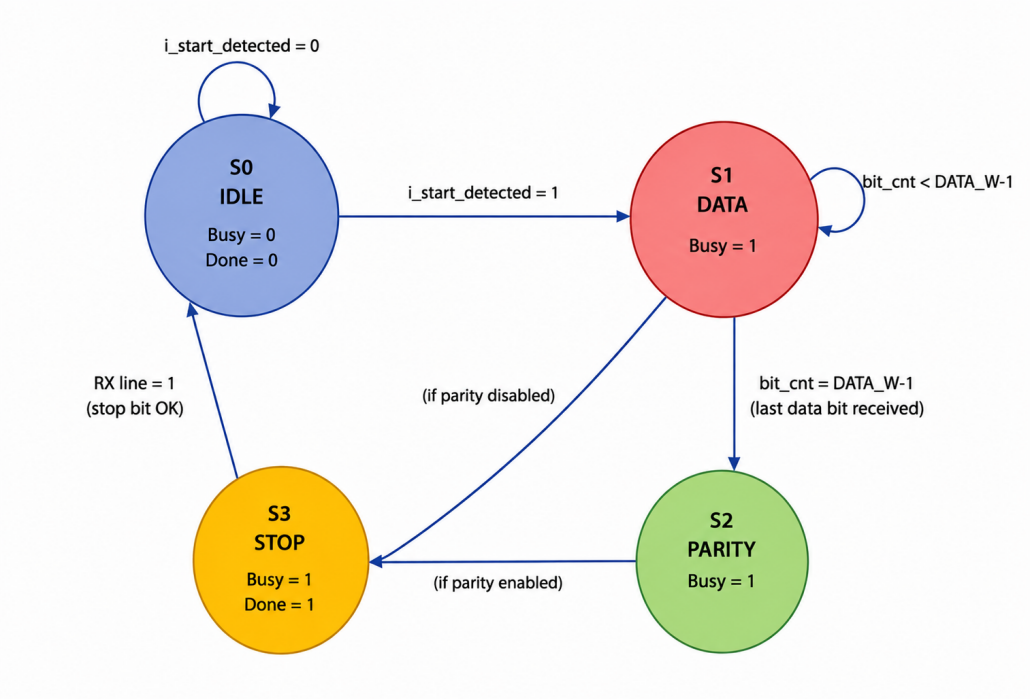


Figure 28: UART Receiver FSM

8.8 Receiver FSM

The `uart_rx_fsm` module controls the receiving process using four states: *idle*, *data*, *parity*, and *stop*. It starts receiving when the start bit is detected, then receives the data bits using a counter. If parity is enabled, it receives the parity bit before checking the stop bit. Finally, it checks the stop bit, indicates that the frame is complete, and returns to the *idle* state.

```
module uart_rx_fsm #(parameter DATA_W = 8) (
 input logic i_clk,
 input logic i_rst_n,
 input logic i_start_detected,
 input logic i_par_en,
 input logic i_rx_bit,
 output logic o_data_en,
 output logic o_parity_en,
 output logic o_stop_en,
 output logic o_busy,
 output logic o_done,
 output logic o_frame_err
);

typedef enum logic [1:0] {idle, data, parity, stop} state_t;
state_t state;
int counter;

always_ff @(posedge i_clk or negedge i_rst_n) begin
    if (i_rst_n == 0) begin
        state <= idle;
        counter <= 0;
        o_busy <= 0;
        o_done <= 0;
        o_frame_err <= 0;
    end
    else begin
        o_done <= 0;

        if (state == idle) begin
            o_busy <= 0;
            if (i_start_detected) begin
                state <= data;
                counter <= 0;
                o_busy <= 1;
            end
```

Figure 29: UART Receiver FSM

8.9 Receiver FSM

The `uart_rx_fsm` module controls the receiving process using four states: *idle*, *data*, *parity*, and *stop*. It starts receiving when the start bit is detected, then receives the data bits using a counter. If parity is enabled, it receives the parity bit before checking the stop bit. Finally, it checks the stop bit, indicates that the frame is complete, and returns to the *idle* state.

```
module uart_rx_fsm #(parameter DATA_W = 8) (
 input logic i_clk,
 input logic i_rst_n,
 input logic i_start_detected,
 input logic i_par_en,
 input logic i_rx_bit,
 output logic o_data_en,
 output logic o_parity_en,
 output logic o_stop_en,
 output logic o_busy,
 output logic o_done,
 output logic o_frame_err
);

typedef enum logic [1:0] {idle, data, parity, stop} state_t;
state_t state;
int counter;

always_ff @(posedge i_clk or negedge i_rst_n) begin
    if (i_rst_n == 0) begin
        state <= idle;
        counter <= 0;
        o_busy <= 0;
        o_done <= 0;
        o_frame_err <= 0;
    end
    else begin
        o_done <= 0;

        if (state == idle) begin
            o_busy <= 0;
            if (i_start_detected) begin
                state <= data;
                counter <= 0;
                o_busy <= 1;
            end
```

Figure 30: UART Receiver FSM

The FSM generates the control signals according to the current state. The `o_data_en`, `o_parity_en`, and `o_stop_en` signals are enabled in their corresponding states, while `o_busy` indicates that the receiver is processing a frame. When the stop bit is checked, `o_done` indicates the end of the frame and `o_frame_err` indicates if the stop bit is incorrect.

```
        end

        if (state == data) begin
            if (counter == DATA_W-1) begin
                counter <= 0;
                if (i_par_en)
                    state <= parity;
                else
                    state <= stop;
            end
            else
                counter <= counter + 1;
        end

        if (state == parity)
            state <= stop;

        if (state == stop) begin
            o_frame_err <= (i_rx_bit != 1);
            o_busy <= 0;
            o_done <= 1;
            state <= idle;
        end
    end
end

always_comb begin
    o_data_en   = (state == data);
    o_parity_en = (state == parity);
    o_stop_en   = (state == stop);
end

endmodule
```

Figure 31: UART Receiver FSM Control Signals

8.10 UART Receiver Top Module

Finally, the `uart_rx` top module connects all the receiver modules together. It mainly handles the connections between the start detector, FSM, serial-to-parallel converter, and parity checker to complete the UART receiving process. Each module performs its own function while the top module combines them to produce the final received data and status signals.

8.11 Parity Bit Capture Issue

During testing, an issue was found in the `uart_rx` top module. The signal `rx_parity_bit` was declared and connected to the parity checker, but it was not assigned any value. This caused the parity checker to compare the calculated parity with an undefined value instead of the actual received parity bit.

To fix this issue, an `always_ff` block was added to capture the parity bit from `rx_bit` when the `parity_en` signal is active.

```
always_ff @(posedge i_clk or negedge i_rst_n) begin
    if(i_rst_n==0)
```

```
        rx_parity_bit <= 1'b0;
    else if(parity_en)
        rx_parity_bit <= rx_bit;
end
```

The `parity_en` signal comes from the FSM and is high for one clock cycle when the parity bit is on the RX line. At this time, the received bit is stored in `rx_parity_bit`. This makes sure that the parity checker receives the actual parity bit. The fix was added in the top module because it already has the clock, reset, `parity_en`, and `rx_bit` signals, so no changes to the parity checker were needed.

8.12 Testbench Results

The grading testbench contains 66 test cases that check the complete UART transmitter and receiver. It tests random data with parity disabled, even and odd parity, parity errors, framing errors, back-to-back frames, and loading data while the transmitter is busy.

The waveform below shows the UART signals during the simulation and confirms that the transmitter and receiver operate correctly.

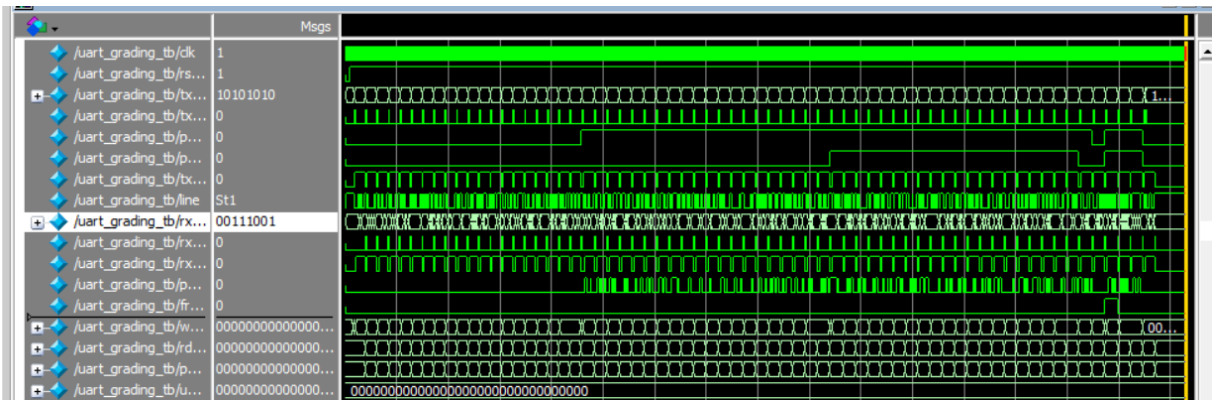


Figure 32: UART Grading Testbench Waveform

The simulation transcript shows that all 66 frames were delivered correctly with no unexpected frames. Therefore, the design achieved a full score of 66/66 and passed the grading testbench.

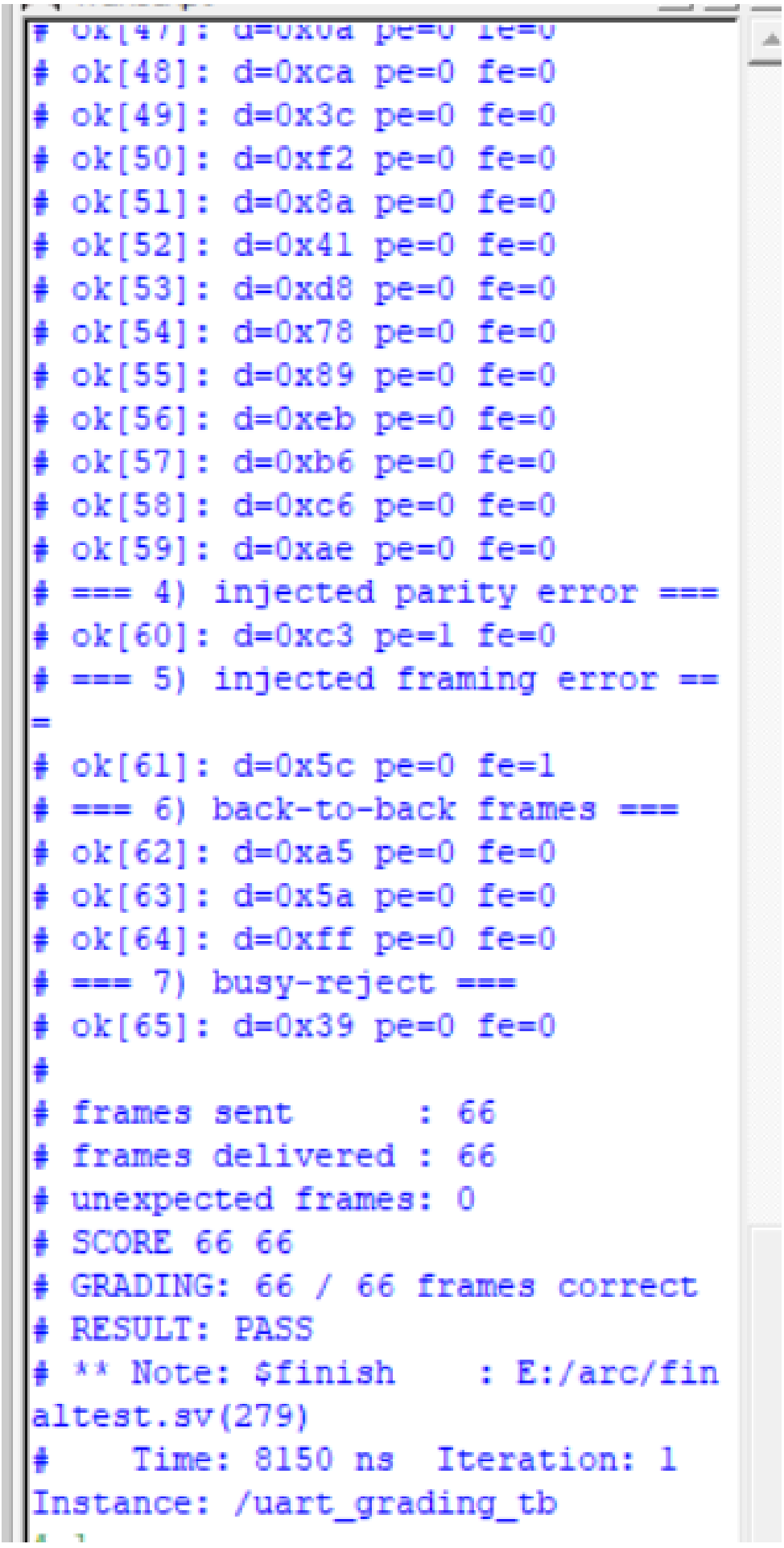


Figure 33: UART Grading Testbench Results